

操作系统原理

第八章：死锁

洪明坚

重庆大学软件学院

February 19, 2019

- ① The deadlock problem
- ② System model
- ③ Methods of handling deadlock
 - Deadlock prevention
 - Deadlock avoidance
 - Deadlock detection and recovery
 - Ostrich algorithm

Outline

- 1 The deadlock problem
- 2 System model
- 3 Methods of handling deadlock
 - Deadlock prevention
 - Deadlock avoidance
 - Deadlock detection and recovery
 - Ostrich algorithm

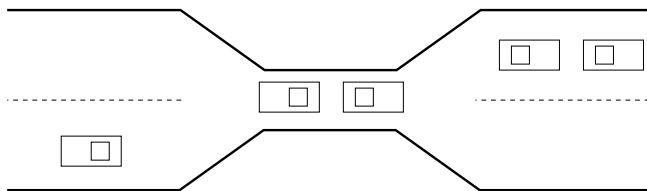
The deadlock problem

- *A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.*
- Example 1
 - The system has 2 tape drives.
 - P_1 and P_2 hold one tape drive and each needs another one.
- Example 2
 - Multi-threaded programs are good candidates for deadlock.
 - Semaphores A and B, initialized to 1.

P_1
P(A);
P(B);

P_2
P(B);
P(A);

Bridge crossing example



- Traffic only in one direction.
- Each section of a bridge can be viewed as a resource.
- If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback).
- Several cars may have to be backed up if a deadlock occurs.
- Starvation is possible.

Outline

- 1 The deadlock problem
- 2 System model
- 3 Methods of handling deadlock
 - Deadlock prevention
 - Deadlock avoidance
 - Deadlock detection and recovery
 - Ostrich algorithm

System model

- A system consists of a finite number of resources to be distributed among a number of competing processes.
- The resources are partitioned into several types.
 - Resource types $R_1, R_2, \dots, R_m$, such as CPU cycles, memory spaces and I/O devices.
- Each resource type R_i has W_i instances
- Actions to use a resource
 - ① **Request** - If the request cannot be granted immediately, then the requesting process must wait until it can acquire the resource;
 - ② **Use** - The process is using the resource;
 - ③ **Release** - The process releases the resources.

Deadlock characterization

- Deadlock can arise if the following four conditions hold *simultaneously*.
 - **Mutual exclusion** - only one process at a time can use a resource.
 - **Hold and wait** - a process holding at least one resource is waiting to acquire additional resources held by other processes.
 - **No preemption** - a resource can be released only *voluntarily* by the process holding it, after that process has completed its task.
 - **Circular wait** - there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , $\dots$, P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

Resource-allocation graph (1/3)

- Deadlocks can be described more precisely in terms of a *directed graph* called *resource-allocation graph*.
- This graph consists of a set of vertices V and a set of edges E .
 - V is partitioned into two types:
 - ① $P = \{P_0, P_1, \dots, P_n\}$, the set consisting of all the processes in the system;
 - ② $R = \{R_0, R_1, \dots, R_m\}$, the set consisting of all resource types in the system.
 - Request edge - directed edge: $P_i \rightarrow R_j$;
 - Assignment edge - directed edge: $R_j \rightarrow P_i$;

Resource-allocation graph (2/3)

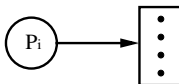
- A process is represented as a circle.



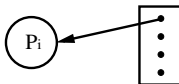
- A resource type with 4 instances.



- P_i requests an instance of R_j .

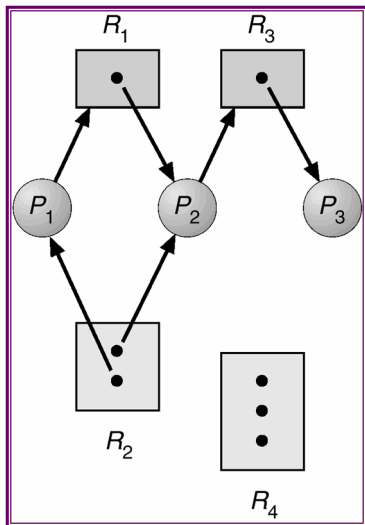


- P_i is holding an instance of R_j .



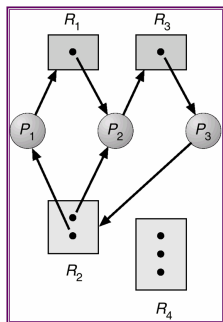
Resource-allocation graph (3/3)

- An example

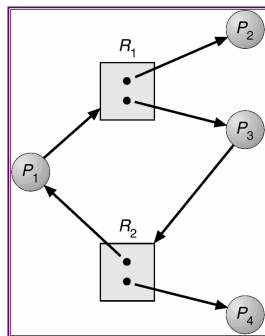


Deadlock and resource-allocation graph

- Given the definition of a resource-allocation graph, it can be shown that
 - If the graph contains no cycles, then no process in the system is deadlocked;
 - If the graph does contain a cycle, then a deadlock *may* exist.



Deadlocked



Not deadlocked

Questions

- Any questions?



Outline

- 1 The deadlock problem
- 2 System model
- 3 Methods of handling deadlock
 - Deadlock prevention
 - Deadlock avoidance
 - Deadlock detection and recovery
 - Ostrich algorithm

Deadlock prevention (1/4)

- As we know, all the four necessary conditions must hold for a deadlock to occur.
 - Deadlock can be prevented by attacking at least one of these conditions.
- Attacking “mutual exclusion”
 - It's not required for sharable resources, such as read-only files; It must hold for non-sharable resources, such as printer.
 - If a device (such as printer) can be spooled
 - Only the printer server uses printer resource (other processes send documents to the printer server to be printed), then deadlock for printer is eliminated.
 - But, not all devices can be spooled.
 - In general, we cannot prevent deadlocks by attacking mutual exclusion: Some resources are intrinsically non-sharable.

Deadlock prevention (2/4)

- Attacking “hold and wait”
 - We must guarantee that whenever a process requests a resource, it does not hold any other resources.
 - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none.
 - May result in low resource utilization and possible starvation.

Deadlock prevention (3/4)

- Attacking “no preemption”
 - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released.
 - This can be applied to resources whose state can be easily saved and restored later, such as CPU.
 - It cannot generally be applied to such resources as printers and tape drives.
- Attacking “circular wait”
 - We can impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration.

Questions

- Any questions?



Deadlock avoidance

- Requires that the system has some additional *a priori* information available.
 - Simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need.
 - The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.
 - Resource-allocation state is defined by the number of available and allocated resources, and the maximum demands of the processes.

Safe state (1/2)

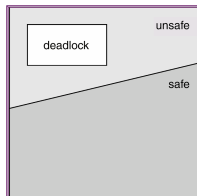
- Definition

- System is in safe state if there exists a safe sequence of all processes $\langle P_0, P_1, \dots, P_n \rangle$ such that for each P_i , the resources that P_i can still request can be satisfied by currently available resources plus resources held by all the P_j , with $j < i$.

- Basic facts

- If a system is in safe state, no deadlocks;
- If a system is in unsafe state, possibility of deadlock.

- Deadlock avoidance must ensure that a system will never enter an unsafe state.



Banker's algorithm

- Banker's algorithm
 - It's invented by Dijkstra in 1968 and used on the **THE** operating system.
 - It can be used in a banking system to ensure that the bank never allocates its available cash such that it can no longer satisfy the needs of all its customers.
- It requires that a new process must declare the maximum number of instances of each resource type that it may need.
- This algorithm requires an order of $m \times n^2$ operations for safety detection.
 - m: The number of resource types
 - n: The number of processes

Banker's algorithm: data structure

- Data structures

- *Available*: Vector of length m . If $available[j] = k$, there are k instances of resource type R_j available.
- *Max*: $n \times m$ matrix. If $Max[i, j] = k$, then process P_i may request at most k instances of resource type R_j .
- *Allocation*: $n \times m$ matrix. If $Allocation[i, j] = k$ then P_i is currently allocated k instances of R_j .
 - $Allocation_i$ means i th row of the *Allocation*.
- *Need*: $n \times m$ matrix. If $Need[i, j] = k$, then P_i may need k more instances of R_j to complete its task.
 - $Need[i, j] = Max[i, j] - Allocation[i, j]$.
 - $Need_i$ means i th row of the *Need*.

Banker's algorithm: Safety detection

- 1 Let *Work* and *Finish* be vectors of length *m* and *n*, respectively. Initialize *Work* = *Available* and *Finish*[*i*] = *false*, for *i* = 1, 2, ..., *n*.
- 2 Find an *i* such that
Finish[*i*] = *false* and $Need_i \leq Work$.
If no such *i* exists, go to step 4.
- 3 $Work = Work + Allocation_i$; *Finish*[*i*] = *true*;
go to step 2.
- 4 If *Finish*[*i*] = *true* for all *i*, then the system is in a safe state.

Banker's algorithm: Resource-request (1/2)

- Let $Request_i$ be the request vector for process P_i .
 - If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j .

Banker's algorithm: Resource-request (2/2)

- 1 If $Request_i \leq Need_i$, go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim.
- 2 If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available.
- 3 Pretend to allocate requested resources to P_i by modifying the state as follows:
 $Available = Available - Request_i;$
 $Allocation_i = Allocation_i + Request_i;$
 $Need_i = Need_i - Request_i;$
- 4 If the system is in safe state, the resources are allocated to P_i ; Otherwise P_i must wait, and the old resource-allocation state is restored.

Banker's algorithm: example (1/3)

- 5 processes and 3 resource types
 - Type A has 10 instance, B has 5 and C has 7.
- Snapshot at time T_0 :

Process	Max			Allocation			Available		
	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	3	3	2
P_1	3	2	2	2	0	0			
P_2	9	0	2	3	0	2			
P_3	2	2	2	2	1	1			
P_4	4	3	3	0	0	2			

Banker's algorithm: example (2/3)

- The content of the matrix *Need* is defined to be $Max - Allocation$.

Process	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria.

Banker's algorithm: example (3/3)

- Assume that P_1 further requests $(1, 0, 2) (\leq (3,3,2))$

Process	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3	2	3	0
P_1	3	2	2	3	0	3	0	2	0			
P_2	9	0	2	3	0	2	6	0	0			
P_3	2	2	2	2	1	1	0	1	1			
P_4	4	3	3	0	0	2	4	3	1			

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement.
 - Can request for $(3,3,0)$ by P_4 be granted?
 - Can request for $(0,2,0)$ by P_0 be granted?

Questions

- Any questions?



Deadlock detection and recovery

- If a system does not employ either a deadlock-prevention or a deadlock-avoidance algorithm, then a deadlock situation may occur.
- In this situation, the operating system must provide:
 - An algorithm to determine whether a deadlock has occurred;
 - An algorithm to recover from the deadlock.

Deadlock detection algorithm

- ➊ Let *Work* and *Finish* be vectors of length m and n , respectively. Initialize $Work = Available$ and if $Allocation_i \neq 0$, then $Finish[i] = false$; otherwise, $Finish[i] = true$.
- ➋ Find an index i such that $Finish[i] = false$ and $Request_i \leq Work$
If no such i exists, go to step 4.
- ➌ $Work = Work + Allocation_i$; $Finish[i] = true$;
go to step 2.
- ➍ If $Finish[i] = false$ for some i ($1 \leq i \leq n$), then the system is in deadlock state.
 - P_i is deadlocked if $Finish[i] = false$.

Deadlock detection algorithm: example (1/2)

- 5 processes and 3 resource types
 - Type A has 7 instance, B has 2 and C has 6.
- Snapshot at time T_0 :

Process	Allocation	Request	Available
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = true$ for all i .

Deadlock detection algorithm: example (2/2)

- Suppose P_2 further requests an additional instance of type C.

Process	Request		
	A	B	C
P_0	0	0	0
P_1	2	0	1
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- State of the system?
 - Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4 .

Deadlock detection algorithm: Usage

- When and how often to invoke deadlock detection algorithm depends on
 - ① How often is a deadlock likely to occur?
 - ② How many processes will be affected by deadlock when it happens?

Questions

- Any questions?



Recovery from deadlock

- When a detection algorithm determines that a deadlock exists, two options are available.
 - ① Process termination
 - Abort all deadlocked processes
 - Abort one process at a time until the deadlock cycle is eliminated.
 - ② Resource preemption
- Both of them are NOT easy.

Questions

- Any questions?



Ostrich algorithm

- We can ignore the problem altogether, and pretend that deadlocks never occur in the system.
 - This solution is used by *most* operating systems, including Windows and Unix/Linux.
 - Because the deadlocks occur very rarely and the deadlock-prevention, deadlock-avoidance or deadlock-detection and recovery algorithms are *costly*.
 - It's a trade-off between *convenience* and *correctness*.

Questions

- Any questions?

